

TLS with server side certificates

Create pem file

<https://www.hivemq.com/docs/hivemq/4.9/user-guide/howtos.html#server>

1. Create a JKS

```
keytool -genkey -keyalg RSA -alias hivemq -keystore hivemq.jks -storepass  
your-keystore-password -validity 360 -keysize 2048
```

2. Export a pem file from the server keystore

```
keytool -exportcert -keystore hivemq.jks -alias hivemq -keypass your-key-password  
-storepass your-keystore-password -rfc -file hivemq-server-cert.pem
```

Remember to change the `-keypass` and `-storepass` passwords accordingly



Setup TLS Listener

1. Navigate to the Configuration folder
.../hivemq-4.10.0/conf/config.xml

Add between `<listeners>` and `</listeners>`

```
<tls-tcp-listener>
  <port>8883</port>
  <bind-address>0.0.0.0</bind-address>
  <name>my-secure-tcp-listener</name>
  <tls>
    <keystore>
      <!-- Configuring the path to the key store -->
      <path>path-to-your-keystore</path>
      <!-- The password of the key store -->
      <password>keystorepassword</password>
      <!-- The password of the private key -->
      <private-key-password>privatekeypassword</private-key-password>
    </keystore>
  </tls>
</tls-tcp-listener>
```



Test TLS Listener

1. Open Terminal and navigate to your folder where the .pem file is located

2. Subscribe to a test topic

```
mqtt sub -h localhost -p 8883 -t test --cafile hivemq-server-cert.pem
```

3. Publish a test message to the test topic

```
mqtt pub -h localhost -p 8883 -t test -m Test --cafile hivemq-server-cert.pem
```



Mutual TLS (mTLS)

mTLS Setup

1. Generate client private key

```
openssl req -x509 -newkey rsa:2048 -nodes -days 360 -keyout mqtt-client-key.pem  
-out mqtt-client-cert.pem
```

2. Convert to crt

```
openssl x509 -outform der -in mqtt-client-cert.pem -out mqtt-client-cert.crt
```

3. Add crt to trusted client keystore

```
keytool -import -file mqtt-client-cert.crt -alias client -keystore  
hivemq-trust-store.jks -storepass changeme -noprompt
```



Broker configuration

1. Navigate to the Configuration folder
.../hivemq-4.10.0/conf/config.xml
2. Add between `</keystore>` and `</tls>`

```
<truststore>  
  <path>path-to-your-truststore</path>  
  <password>changeme</password>  
</truststore>
```



Test mTLS encryption

- Start the Broker with the new configuration
- Open 2 Terminal windows
- From the folder, where the certificates are located

```
mqtt sub -t test -p 8883 --cert mqtt-client-cert.pem --key mqtt-client-key.pem  
--cafile hivemq-server-cert.pem
```

- On the other Terminal window, publish a test message

```
mqtt pub -t test -p 8883 --cert mqtt-client-cert.pem --key mqtt-client-key.pem  
--cafile hivemq-server-cert.pem -m 'This message is encrypted'
```



X.509 and ESE

mTLS Setup

1. Repeat steps from slide 6 and create x.509 certificates and Public Keys for users with Common Name (CN)
 - frontendclients
 - backendservice
2. Add both crts to trusted client keystore



Configure PostgreSQL

Configure PostgreSQL from CLI

Pull docker container

```
docker pull postgres
```

Run docker container

```
docker run --name postgres -e POSTGRES_PASSWORD=password -d postgres
```

Login from CLI

```
psql -U postgres -W
```

```
Password: password
```

Configure PostgreSQL from CLI

Create database

```
CREATE DATABASE ese;  
\c ese;  
Password: password
```

Go to your HiveMQ installation folder and copy the content from file
.../hivemq-4.10.0/extensions/hivemq-enterprise-security-extension/scripts/sql/postgresql_create.sql

Paste the content into your CLI and check for success

```
\d
```

```
ese=# \d  
public | cc_permissions | table | postgres  
public | cc_permissions_id_seq | sequence | postgres  
public | cc_role_permissions | table | postgres  
public | cc_roles | table | postgres  
public | cc_roles_id_seq | sequence | postgres  
public | cc_user_permissions | table | postgres  
public | cc_user_roles | table | postgres  
public | cc_users | table | postgres  
public | cc_users_id_seq | sequence | postgres  
public | permissions | table | postgres  
public | permissions_id_seq | sequence | postgres  
public | rest_api_permissions | table | postgres  
public | rest_api_permissions_id_seq | sequence | postgres  
public | rest_api_role_permissions | table | postgres  
public | rest_api_roles | table | postgres  
public | rest_api_roles_id_seq | sequence | postgres  
public | rest_api_user_permissions | table | postgres  
public | rest_api_user_roles | table | postgres  
public | rest_api_users | table | postgres  
public | rest_api_users_id_seq | sequence | postgres  
public | role_permissions | table | postgres  
public | roles | table | postgres  
public | roles_id_seq | sequence | postgres  
public | user_permissions | table | postgres  
public | user_roles | table | postgres  
public | users | table | postgres  
public | users_id_seq | sequence | postgres
```



Configure PostgreSQL from CLI

Enter the following on your CLI:

```
insert into public.users
(username, password, password_iterations, password_salt, algorithm)
values
('backendservice', 'wtUo2dri+ttHGHRpngg9uG21piWLiKSX7IaNSnU/BfN9pt+ZOLQByG/3JlPPQ7t/p18S3tjR2+Um/DPBdAQUlg==', 100, 'Nv6NU9XY7tvHdSGaKmNTow==', 'SHA512'),
('frontendclient', 'ZHg/rNJel1BH0YMEvc40ekCRUE5vVLcsPF6mk9GPDcdEmX3stm50MplaqjGb8Lxhy6rNFQZSQRsb0xmFZ8ps1Q==', 100, 'JhpW27QU9WfIaG6FJT5MkQ==', 'SHA512'),
('superuser', 'n0gr9xVnkt51Lr6KS/rAKm/LqAt8oEk17vCerRod3qDbyMFfDBGt8obnkW+AGygxCQDWdaA2sQnXxoAbVK6Q==', 100, 'wxw+3d1CV4bWXQhb6LLniA==', 'SHA512');

insert into public.permissions
(id, topic, publish_allowed, subscribe_allowed, qos_0_allowed, qos_1_allowed, qos_2_allowed, retained_msgs_allowed, shared_sub_allowed, shared_group)
values
(1, 'topic/+status', false, true, true, true, true, false, false, ''),
(2, 'topic/${mqtt-clientid}/status', true, false, true, true, true, true, false, ''),
(3, '#', true, true, true, true, true, true, true, '');

insert into public.roles
(id, name, description)
values
(1, 'backendservice', 'only allowed to subscribe to topics'),
(2, 'frontendclients', 'only allowed to publish to topics'),
(3, 'superuser', 'is allowed to do everything');

insert into public.user_roles
(user_id, role_id)
values
(1, 1),
(2, 2),
(3, 3);

insert into public.role_permissions
(role, permission)
values
(1, 1),
(2, 2),
(3, 3);
```



Configure PostgreSQL from CLI

```
ese=# insert into public.users
(username, password, password_iterations, password_salt, algorithm)
values
('backendservice', 'wtUo2dri+ttHGHRpngg9uG21piWLiKSX7IaNSnU/Bfn9pt+ZOLQByG/3JlPPQ7t/pl8S3tjR2+Um/DPBdAQLg==', 100, 'Nv6NU9XY7tvHdSGaKmNTow==', 'SHA512'),
('frontendclient', 'Zhg/rNJe11BH0YMEvc40ekCRUE5vLcsPF6mk9GPDcdEmX3stm50MplaqjGb8Lxhy6rNFQZSQRsboXmfZ8ps1Q==', 100, 'JhpW27QU9WfIaG6FJTSmkQ==', 'SHA512'),
('superuser', 'n0gr9xVnkt51Lr68KS/rAKm/LqxAt8oEki7vCernRod3qDbyMFFDBGT8obnkW+AGygxCQDWdaA2sQnXXoAbVK6Q==', 100, 'wxw+3diCV4bWXQh6LLnIA==', 'SHA512');

insert into public.permissions
(id, topic, publish_allowed, subscribe_allowed, qos_0_allowed, qos_1_allowed, qos_2_allowed, retained_msgs_allowed, shared_sub_allowed, shared_group)
values
(1, 'topic/+status', false, true, true, true, true, false, false, ''),
(2, 'topic/${mqtt-clientid}/status', true, false, true, true, true, true, false, ''),
(3, '#', true, true, true, true, true, true, true, '');

insert into public.roles
(id, name, description)
values
(1, 'backendservice', 'only allowed to subscribe to topics'),
(2, 'frontendclients', 'only allowed to publish to topics'),
(3, 'superuser', 'is allowed to do everything');

insert into public.user_roles
(user_id, role_id)
values
(1, 1),
(2, 2),
(3, 3);

insert into public.role_permissions
(role, permission)
values
(1, 1),
(2, 2),
(3, 3);

INSERT 0 3
INSERT 0 3
INSERT 0 3
INSERT 0 3
INSERT 0 3
ese=# select * from users;
 1 | backendservice | wtUo2dri+ttHGHRpngg9uG21piWLiKSX7IaNSnU/Bfn9pt+ZOLQByG/3JlPPQ7t/pl8S3tjR2+Um/DPBdAQLg== | 100 | Nv6NU9XY7tvHdSGaKmNTow== | SHA512 | 2022-12-14 11:52:45.101603+01 | 2022-12-14 11:52:45.101603+01
 2 | frontendclient | Zhg/rNJe11BH0YMEvc40ekCRUE5vLcsPF6mk9GPDcdEmX3stm50MplaqjGb8Lxhy6rNFQZSQRsboXmfZ8ps1Q== | 100 | JhpW27QU9WfIaG6FJTSmkQ== | SHA512 | 2022-12-14 11:52:45.101603+01 | 2022-12-14 11:52:45.101603+01
 3 | superuser      | n0gr9xVnkt51Lr68KS/rAKm/LqxAt8oEki7vCernRod3qDbyMFFDBGT8obnkW+AGygxCQDWdaA2sQnXXoAbVK6Q== | 100 | wxw+3diCV4bWXQh6LLnIA== | SHA512 | 2022-12-14 11:52:45.101603+01 | 2022-12-14 11:52:45.101603+01
```



Configure Enterprise Security Extension

Configure Enterprise Security Extension

Go to the Configuration file of your Enterprise Security Extension

hivemq-4.10.0/extensions/hivemq-enterprise-security-extension/conf/enterprise-security-extension.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<enterprise-security-extension
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="enterprise-security-extension.xsd"
  version="1">
  <realms>
    <sql-realm>
      <name>postgres-backend</name>
      <enabled>true</enabled>
      <configuration>
        <db-type>POSTGRES</db-type>
        <db-name>ese</db-name>
        <db-host>localhost</db-host>
        <db-port>5432</db-port>
        <db-username>postgres</db-username>
        <db-password>password</db-password>
      </configuration>
    </sql-realm>
  </realms>
  <pipelines>
    <!-- secure access to the mqtt broker -->
    <listener-pipeline listener="ALL">
      <authentication-preprocessors>
        <x509-preprocessor prefix="{{" postfix="}}">
          <x509-extractions>
            <x509-extraction>
              <x509-field>subject-common-name</x509-field>
              <ese-variable>authentication-key</ese-variable>
            </x509-extraction>
          </x509-extractions>
        </x509-preprocessor>
      </authentication-preprocessors>
    </listener-pipeline>
  </pipelines>
</enterprise-security-extension>
```

```
<logging-preprocessor>
  <message>Key = ${authentication-key}</message>
  <level>info</level>
  <name>com.example.logger</name>
</logging-preprocessor>
</authentication-preprocessors>
<allow-all-authentication-manager />
<authorization-preprocessors>
  <plain-preprocessor>
    <transformations>
      <transformation encoding="UTF8">
        <from>authentication-key</from>
        <to>authorization-role-key</to>
      </transformation>
    </transformations>
  </plain-preprocessor>
  <logging-preprocessor>
    <message>Key = ${authorization-key} - Role Key = ${authorization-role-key}</message>
    <level>info</level>
    <name>com.example.logger</name>
  </logging-preprocessor>
</authorization-preprocessors>
<sql-authorization-manager>
  <realm>postgres-backend</realm>
  <use-authorization-key>false</use-authorization-key>
  <use-authorization-role-key>true</use-authorization-role-key>
</sql-authorization-manager>
</listener-pipeline>
</pipelines>
</enterprise-security-extension>
```



Configure Enterprise Security Extension

Download JDBC driver from

<https://jdbc.postgresql.org/download/>

And move the jar file to

.../hivemq-4.10.0/extensions/hivemq-enterprise-security-extension/drivers/jdbc/postgresql-42.5.1.jar

Remove the DISABLED file from your installation folder of Enterprise Security Extension

.../hivemq-4.10.0/extensions/hivemq-enterprise-security-extension/DISABLED

Start the broker

.../hivemq-4.10.0/bin/run.sh



Test

- Open 2 Terminal windows
- From the folder, where the certificates are located

```
mqtt sub -t "topic/+/status" -p 8883 --cert backendservice.pem --key  
backendservicekey.pem --cafile hivemq-server-cert.pem
```

- On the other Terminal window, publish a test message

```
mqtt pub -t "topic/frontendclients/status" -p 8883 --cert frontendclients.pem  
--key frontendclientskey.pem --cafile hivemq-server-cert.pem -m "This message is  
encrypted" -i frontendclients
```

If you copy paste the commands above, the quotation marks "" might not format correctly, please make sure to replace the quotation marks in your CLI or use an MQTT client

